



Certified Kubernetes Administrator (CKA): Managing Cluster Lifecycle and Upgrades

Backing up and restoring etcd for disaster recovery



Tim Warner

Principal Author | IT Ops | Pluralsight

TechTrainerTim.com



Four framing questions



What does etcd database actually store, and why is it the single source of truth?



How do I take an etcd snapshot, and which binary does the work?



What's the full restore workflow when disaster strikes?



How do stacked and external etcd topologies compare?





"It's 2 a.m., and the on-call engineer just deleted the production namespace. Deployments, Services, ConfigMaps -- all gone. The CTO is on Slack with one question: do we have a backup?"



**What does etcd store, and why is it the
single source of truth?**



etcd: The cluster's single source of truth

Every Kubernetes object is serialized and stored in etcd

Deployments, Services, ConfigMaps, Secrets, RBAC rules -- all of it

etcd is a distributed key-value store that lives in the control plane



etcd: The cluster's single source of truth

Lose etcd without a backup and the cluster's
declared state is gone

etcdctl speaks the v3 API, the default since
etcd 3.4

Only the API server reads and writes etcd
directly



**How do I take an etcd snapshot, and
which binary does the work?**



Taking an etcd snapshot with etcdctl

`--endpoints=https://127.0.0.1:2379`

- targets the local etcd

`--cacert`

- verifies the etcd server's identity

`etcdctl snapshot save`

- takes a live, point-in-time backup

`save`

- stays in etcdctl because it talks to a running cluster

`Verify the file`

- with `etcdctl snapshot status`, not `etcdctl`

`--cert and --key`

- are your client credentials for mutual TLS



```
# Take a live snapshot -- ONLINE op, stays in etcdctl, needs mutual TLS
ETCDCTL_API=3 etcdctl snapshot save /tmp/etcd-backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Verify the file -- OFFLINE op, moved to etcdutl in etcd 3.6
etcdutl snapshot status /tmp/etcd-backup.db --write-out=table
```

Snapshot save with etcdctl, verify with etcdutl



What's the full restore workflow when disaster strikes?



The etcd restore workflow, step by step

Stop the API server:
move kube-
apiserver.yaml out of
/etc/kubernetes/manifests/

Restore to a NEW data
directory with etcdctl
snapshot restore

Point etcd.yaml's
hostPath at the
restored directory

Move kube-
apiserver.yaml back so
kubelet restarts it

Verify with kubectl get
nodes, then confirm the
deleted objects are
back



Restore from a snapshot with etcdctl

1) Stop the API server (static Pod) by moving its manifest aside

```
sudo mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/
```

2) Restore into a NEW data dir -- OFFLINE op, etcdctl (no TLS flags)

```
sudo etcdctl snapshot restore /tmp/etcd-backup.db \  
  --data-dir=/var/lib/etcd-restored
```

3) Repoint etcd at the restored data dir (the step folks forget)

```
sudo sed -i 's#/var/lib/etcd$/var/lib/etcd-restored#' \  
  /etc/kubernetes/manifests/etcd.yaml
```

4) Bring the API server back; kubelet restarts etcd + apiserver

```
sudo mv /tmp/kube-apiserver.yaml /etc/kubernetes/manifests/
```

5) Confirm the cluster and the recovered objects

```
kubectl get nodes
```

```
kubectl get all -n globomantics-shop
```



How do stacked and external etcd topologies compare?



Stacked vs. external etcd topologies

Stacked

Stacked is kubeadm's default -- fewer nodes, simpler to stand up

Stacked couples failure: lose the node, lose a control-plane AND etcd member

Stacked: etcd runs on each control-plane node, next to the API server

vs.

External

External isolates the etcd failure domain and is more resilient

External costs more nodes to provision, patch, monitor, and back up

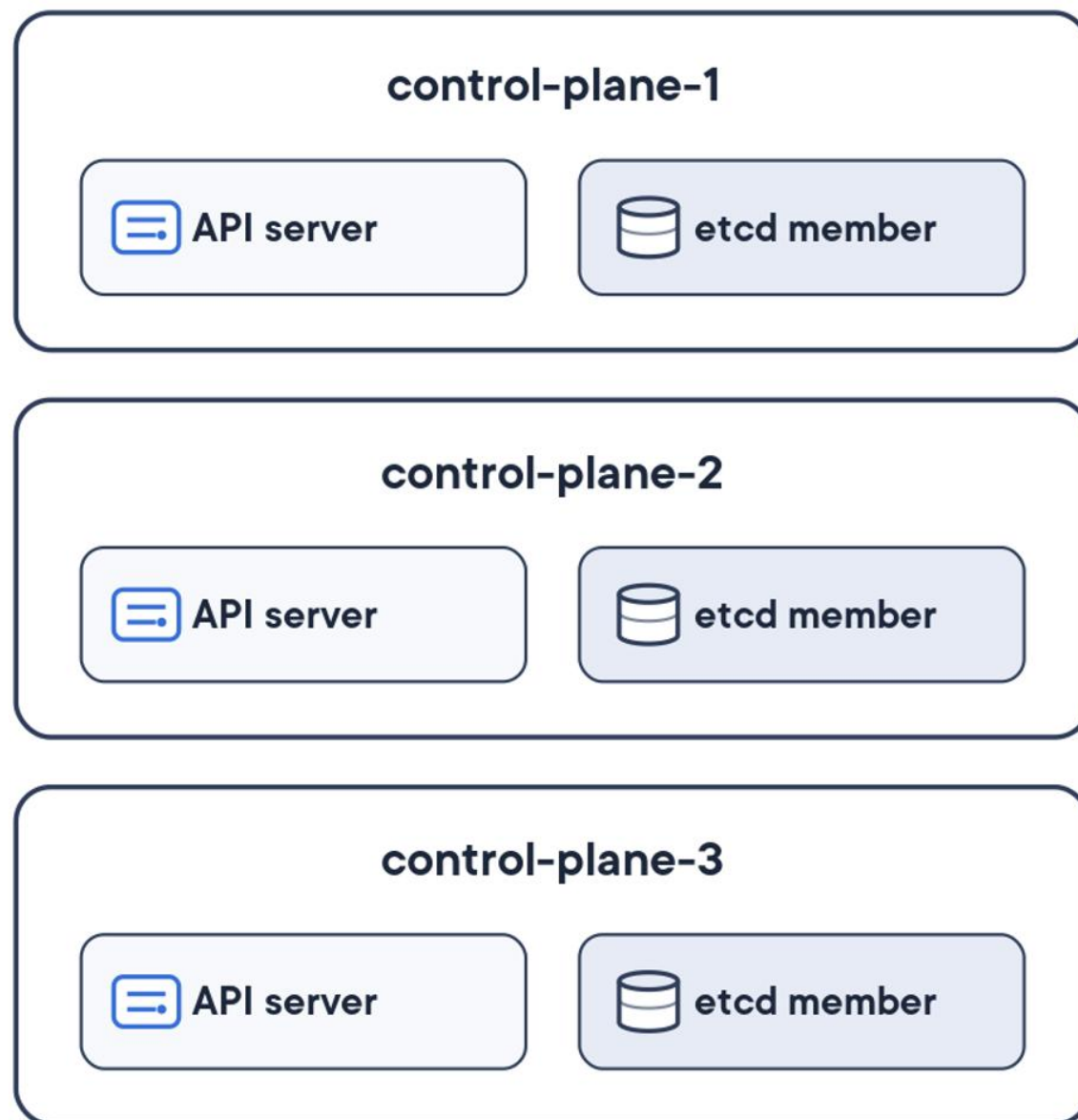
External: etcd runs on its own dedicated nodes, off the control plane



Stacked vs. external etcd: one picture

STACKED

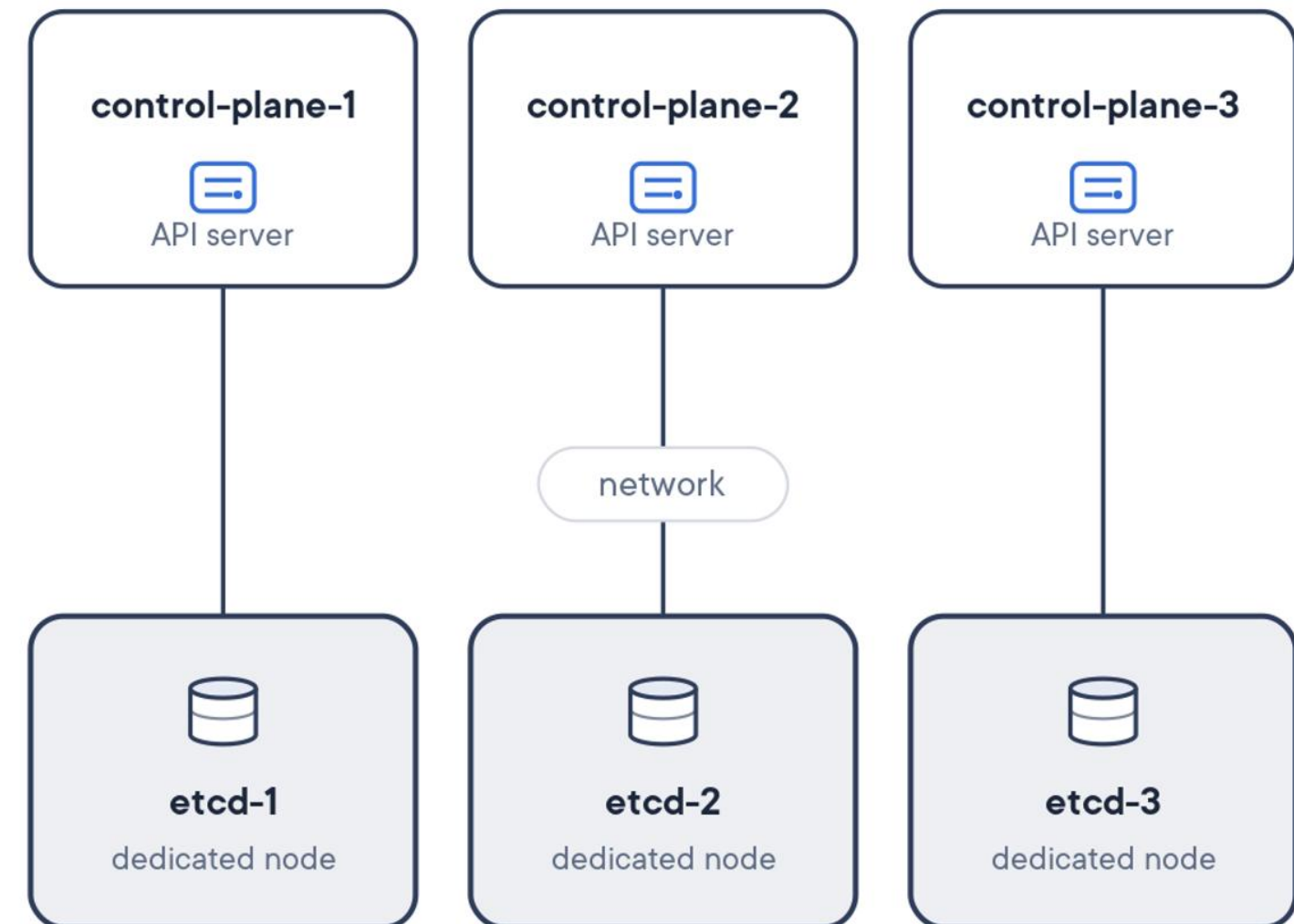
kubeadm default



Simpler, fewer nodes -- but a lost node takes a control-plane AND an etcd member with it.

EXTERNAL

dedicated etcd nodes



Resilient -- isolates the etcd failure domain -- but you provision, patch, and back up more nodes.



Demo

Back up, destroy, and restore etcd

Snapshot the live Vagrant VM cluster, delete a namespace, and bring it back with `etcdctl` and `etcdutl`.





"The restore took four minutes. Four minutes from total disaster to a healthy cluster. The CTO stopped asking whether we had a backup and started asking how we put one on every cluster."





From Globomantics to you



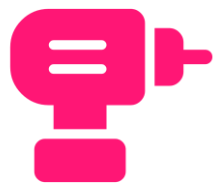
etcd is the single source of truth

protect it or you can't recover



snapshot save lives in etcdctl

status and restore moved to etcdutil in 3.6



Drill the restore

stop API server, restore to a new dir, repoint etcd, verify



Know both topologies

stacked is simple, external isolates failure

